

Project: Epidemie su reti

diffusione epidemica, grafi e metodi computazionali

1. Obiettivi della dispensa

Questa dispensa introduce i modelli epidemici su reti come caso di studio centrale per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono sei:

1. rappresentare una popolazione come grafo di contatti;
2. formulare dinamiche SI, SIS, SIR e SEIR su rete;
3. derivare una chiusura mean-field a livello di nodo;
4. discutere la stabilit  dello stato disease-free e la soglia epidemica;
5. introdurre una estensione strutturata per et  tramite matrici di contatto;
6. preparare il passaggio a simulazioni Monte Carlo e a interventi come vaccinazione e isolamento selettivo.

Dal punto di vista didattico, questo modulo   particolarmente forte perch  collega in modo naturale:

- rappresentazione di reti;
- sistemi di ODE;
- criteri spettrali;
- simulazioni agent-based;
- analisi di intervento.

2. Motivazione generale

I modelli epidemici classici a mescolamento omogeneo assumono che ogni individuo possa incontrare qualunque altro individuo con la stessa probabilit . Questa ipotesi   utile come prima approssimazione, ma spesso troppo forte.

In molte situazioni reali i contatti sono strutturati:

- famiglie;
- scuole;
- luoghi di lavoro;
- reti di trasporto;
- contatti digitali;

- interazioni per classi di età.

Rappresentare la popolazione come rete permette di incorporare questa eterogeneità strutturale. L'infezione non si diffonde più in uno spazio omogeneo, ma lungo archi di contatto specifici.

3. Rappresentazione della rete

Consideriamo una rete con N nodi, dove ogni nodo rappresenta un individuo. La struttura dei contatti è descritta dalla matrice di adiacenza

$$A = (a_{ij}),$$

dove

$$a_{ij} = \begin{cases} 1 & \text{se esiste un contatto tra } i \text{ e } j, \\ 0 & \text{altrimenti.} \end{cases}$$

Nel caso più semplice assumiamo una rete non orientata, quindi

$$a_{ij} = a_{ji}.$$

Il grado del nodo i è

$$k_i = \sum_{j=1}^N a_{ij}.$$

Questa quantità misura il numero di contatti del nodo i .

4. Modello SI su rete

4.1 Stati del modello

Nel modello SI ogni individuo può trovarsi in uno di due stati:

- suscettibile S ;
- infetto I .

Non esiste guarigione. Una volta infetto, il nodo resta infetto per sempre.

4.2 Dinamica microscopica

Se il nodo i e' suscettibile, puo' diventare infetto a causa dei suoi vicini infetti. Se β e' il tasso di trasmissione per contatto, allora l'intensita' di infezione cresce con il numero di vicini infetti.

Una descrizione media a livello di nodo usa la probabilita' $p_i(t)$ che il nodo i sia infetto al tempo t .

Nella chiusura mean-field a livello di nodo, la dinamica SI diventa

$$\dot{p}_i = \beta(1 - p_i) \sum_{j=1}^N a_{ij} p_j.$$

Interpretazione:

- $(1 - p_i)$ e' la probabilita' che il nodo sia ancora suscettibile;
- la somma sui vicini misura la pressione infettiva locale.

5. Modello SIS su rete

5.1 Stati del modello

Nel modello SIS gli individui possono essere:

- suscettibili S ;
- infetti I .

Ma, a differenza del caso SI, un individuo infetto puo' guarire e tornare suscettibile.

Se μ e' il tasso di guarigione, la dinamica mean-field a livello di nodo e'

$$\dot{p}_i = -\mu p_i + \beta(1 - p_i) \sum_{j=1}^N a_{ij} p_j.$$

Questa e' una delle equazioni piu' importanti dell'intera dispensa.

5.2 Significato dei due termini

Il primo termine

$$-\mu p_i$$

descrive la perdita di probabilita' di infezione dovuta alla guarigione.

Il secondo termine

$$\beta(1 - p_i) \sum_j a_{ij} p_j$$

descrive la probabilita' di nuova infezione dovuta ai vicini infetti.

Il modello SIS e' particolarmente importante perche' ammette in generale uno stato disease-free e, in certe condizioni, uno stato endemico persistente.

6. Modello SIR su rete

6.1 Stati del modello

Nel modello SIR ogni nodo puo' essere in uno dei tre stati:

- suscettibile S ;
- infetto I ;
- rimosso o guarito R .

Un individuo infetto guarisce con tasso μ e non torna piu' suscettibile.

6.2 Descrizione a livello di nodo

Indichiamo con:

- $s_i(t)$ la probabilita' che il nodo i sia suscettibile;
- $p_i(t)$ la probabilita' che sia infetto;
- $r_i(t)$ la probabilita' che sia rimosso.

Con vincolo

$$s_i(t) + p_i(t) + r_i(t) = 1.$$

La dinamica mean-field e'

$$\dot{s}_i = -\beta s_i \sum_{j=1}^N a_{ij} p_j,$$

$$\dot{p}_i = \beta s_i \sum_{j=1}^N a_{ij} p_j - \mu p_i,$$

$$\dot{r}_i = \mu p_i.$$

Questo modello e' adatto a malattie che conferiscono immunita' dopo il contagio o dopo la guarigione.

7. Modello SEIR su rete

7.1 Stati del modello

Nel modello SEIR si aggiunge uno stato esposto E :

- S suscettibile;
- E esposto ma non ancora infettivo;
- I infetto;
- R rimosso o guarito.

Se σ e' il tasso di progressione da esposto a infetto, la dinamica a livello di nodo diventa

$$\dot{s}_i = -\beta s_i \sum_{j=1}^N a_{ij} p_j,$$

$$\dot{e}_i = \beta s_i \sum_{j=1}^N a_{ij} p_j - \sigma e_i,$$

$$\dot{p}_i = \sigma e_i - \mu p_i,$$

$$\dot{r}_i = \mu p_i.$$

Questa estensione e' utile quando esiste una fase latente tra infezione e contagiosita'.

8. Mean-field a livello di nodo

8.1 Perche' serve una chiusura

La dinamica esatta su rete richiederebbe di tenere traccia delle correlazioni tra stati dei nodi vicini. Questo diventa rapidamente molto complesso.

La chiusura mean-field a livello di nodo assume, in sostanza, che le probabilita' dei diversi nodi possano essere trattate in modo fattorizzato. Questo permette di ottenere un sistema chiuso di ODE per le probabilita' marginali $p_i(t)$.

8.2 Vantaggi e limiti

Vantaggi:

- il sistema resta trattabile numericamente;
- la rete entra ancora in modo esplicito tramite la matrice A ;
- la struttura eterogenea del grafo viene mantenuta.

Limiti:

- si trascurano correlazioni di ordine superiore;
- il modello puo' sovra- o sotto-stimare la dinamica in reti fortemente strutturate;
- le approssimazioni peggiorano quando clustering e dipendenze locali sono forti.

Nonostante questo, il mean-field a livello di nodo e' spesso una base eccellente per analisi qualitative e computazionali.

9. Stato disease-free e stabilita'

9.1 Stato disease-free

Nel modello SIS, lo stato disease-free corrisponde a

$$p_i = 0 \quad \text{per ogni } i.$$

Nel modello SIR o SEIR, la nozione analoga si riferisce a una configurazione in cui non ci sono nodi infetti.

9.2 Linearizzazione del modello SIS

Per studiare la stabilita' del disease-free, si linearizza la dinamica SIS vicino a $p_i = 0$. Poiche' per probabilita' piccole vale approssimativamente

$$1 - p_i \approx 1,$$

si ottiene

$$\dot{p}_i \approx -\mu p_i + \beta \sum_{j=1}^N a_{ij} p_j.$$

In forma vettoriale:

$$\dot{p} \approx (\beta A - \mu I)p.$$

Questa e' la formula centrale per l'analisi della soglia epidemica.

10. Criterio spettrale e soglia epidemica

10.1 Ruolo dell'autovalore principale

Sia $\lambda_{\max}(A)$ il massimo autovalore della matrice di adiacenza. Lo stato disease-free e' stabile se tutte le componenti della dinamica lineare decadono, cioe' se

$$\beta\lambda_{\max}(A) - \mu < 0.$$

Equivalentemente,

$$\frac{\beta}{\mu} < \frac{1}{\lambda_{\max}(A)}.$$

Questa e' la soglia epidemica nel modello mean-field su rete.

10.2 Interpretazione

Il rapporto

$$\frac{\beta}{\mu}$$

misura il bilancio tra contagio e guarigione.

Il termine

$$\lambda_{\max}(A)$$

misura invece la capacita' strutturale della rete di sostenere la propagazione.

Quindi la soglia dipende da due elementi:

- intensita' biologica o comportamentale del contagio;
- topologia della rete.

Questo e' un punto fondamentale dell'intero modulo: la diffusione epidemica non dipende solo dai parametri clinici, ma anche dalla struttura dei contatti.

11. Interpretazione della soglia

Se

$$\frac{\beta}{\mu} < \frac{1}{\lambda_{\max}(A)},$$

piccole infezioni iniziali tendono a spegnersi.

Se invece

$$\frac{\beta}{\mu} > \frac{1}{\lambda_{\max}(A)},$$

la malattia puo' invadere la rete e, nel caso SIS, sostenere uno stato endemico.

Dal punto di vista computazionale, questo rende naturale un confronto tra:

- predizione spettrale;
- simulazioni numeriche del sistema mean-field;
- simulazioni agent-based stocastiche.

12. Struttura per eta' e matrici di contatto

12.1 Perche' introdurre classi di eta'

In molte epidemie, la probabilita' di contatto e la suscettibilita' non sono omogenee. Bambini, adulti e anziani possono avere pattern di contatto molto diversi.

Per rappresentare questa eterogeneita', si introducono classi di eta' e una matrice di contatto

$$C = (c_{\alpha\beta}),$$

dove $c_{\alpha\beta}$ misura l'intensita' media dei contatti da individui del gruppo α verso individui del gruppo β .

12.2 Modello strutturato

Se $S_\alpha(t)$, $I_\alpha(t)$, $R_\alpha(t)$ rappresentano le frazioni nelle diverse classi, un modello SIR strutturato per eta' puo' assumere la forma

$$\dot{S}_\alpha = -\beta S_\alpha \sum_{\beta} c_{\alpha\beta} I_\beta,$$

$$\dot{I}_\alpha = \beta S_\alpha \sum_{\beta} c_{\alpha\beta} I_\beta - \mu I_\alpha,$$

$$\dot{R}_\alpha = \mu I_\alpha.$$

Questa e' una generalizzazione molto importante, perche' collega la dinamica epidemica a dati empirici di contatto.

12.3 Significato epidemiologico

La matrice di contatto consente di studiare:

- scuole e chiusure scolastiche;
- protezione selettiva degli anziani;
- vaccinazione mirata per gruppi;
- regioni sicure o safe regions con ridotta connettività.

Questa estensione e' particolarmente utile per discutere interventi.

13. Vaccinazione e interventi strutturali

13.1 Vaccinazione

In una forma elementare, la vaccinazione riduce la frazione di suscettibili o riduce la trasmissibilita' effettiva.

Per esempio, nel modello strutturato per eta', si puo' introdurre una copertura vaccinale v_α e sostituire

$$S_\alpha(0)$$

con

$$(1 - v_\alpha)S_\alpha(0).$$

Questo modifica la dinamica e puo' spostare il sistema al di sotto della soglia epidemica.

13.2 Riduzione dei contatti

Un secondo tipo di intervento agisce sulla rete o sulla matrice di contatto:

- riduzione dei contatti scolastici;
- isolamento di nodi ad alto grado;
- limitazione di archi tra gruppi;
- creazione di zone relativamente protette.

Dal punto di vista matematico, questi interventi modificano direttamente A o C , e quindi anche il criterio spettrale di invasione.

14. Confronto tra livello macroscopico e simulazione agent-based

I modelli mean-field producono ODE per probabilita' o frequenze attese. Le simulazioni agent-based, invece, lavorano a livello di eventi discreti:

- un nodo si infetta;

- un nodo guarisce;
- un contatto attiva una trasmissione.

Questo confronto e' molto istruttivo per gli studenti, perche' mostra la differenza tra:

- dinamica media deterministica;
- realizzazioni stocastiche su una rete finita.

15. Pseudocodice del modello SIS su rete

Input

- matrice di adiacenza A
- tasso di contagio β
- tasso di guarigione μ
- condizioni iniziali $p_i(0)$
- passo temporale Δt
- numero di iterazioni T

Pseudocodice

1. inizializza il vettore delle probabilita' $p(0)$
2. per ogni tempo:
 - per ogni nodo i :
 - calcola la pressione infettiva

$$h_i = \sum_j a_{ij} p_j$$

– aggiorna

$$p_i \leftarrow p_i + \Delta t [-\mu p_i + \beta(1 - p_i)h_i]$$

- salva il vettore aggiornato
3. restituisci la traiettoria

Questo e' il punto di partenza piu' naturale per un laboratorio computazionale.

16. Pseudocodice di una simulazione agent-based

Per una simulazione discreta stocastica del contagio su rete:

1. ogni nodo ha uno stato S , I , R oppure E ;
2. a ogni passo si controllano i vicini infetti;
3. ogni nodo suscettibile puo' infettarsi con probabilita' dipendente dai vicini;
4. ogni nodo infetto puo' guarire con una certa probabilita';
5. si aggiorna lo stato di tutti i nodi.

Questa struttura si presta molto bene a simulazioni Monte Carlo e confronto con il modello mean-field.

17. Schema del laboratorio

17.1 Laboratorio 1 - SI e SIS su piccoli grafi

Obiettivo

Capire il ruolo della struttura della rete.

Attività

1. costruire piccoli grafi semplici;
2. simulare il modello SI;
3. simulare il modello SIS;
4. confrontare l'effetto di grado, centralità e connettività.

Domande guida

- quali nodi si infettano per primi?
- la struttura del grafo cambia molto la velocità di diffusione?
- il modello SIS produce uno stato endemico?

17.2 Laboratorio 2 - Soglia epidemica e autovalore principale

Obiettivo

Verificare il criterio spettrale.

Attività

1. calcolare $\lambda_{\max}(A)$ per reti diverse;
2. variare β/μ ;
3. integrare il sistema mean-field;
4. confrontare il comportamento sopra e sotto soglia.

Domande guida

- il criterio predice bene l'invasione?
- quali reti hanno soglia più bassa?
- la rete a hub è più vulnerabile?

17.3 Laboratorio 3 - SIR e SEIR

Obiettivo

Studiare la differenza tra infezione permanente, reinfezione e immunità.

Attività

1. implementare SI, SIS, SIR e SEIR;
2. confrontare prevalenza, picco epidemico e durata;

3. variare σ nel modello SEIR;
4. interpretare epidemiologicamente le differenze.

Domande guida

- quando compare un picco epidemico?
- che effetto ha la fase esposta?
- in quali modelli il contagio si estingue automaticamente?

17.4 Laboratorio 4 - Matrici di contatto e interventi

Obiettivo

Studiare l'effetto di eterogeneità per età e interventi mirati.

Attività

1. costruire una matrice di contatto semplice;
2. simulare il modello strutturato;
3. introdurre vaccinazione selettiva;
4. ridurre alcuni contatti per simulare safe regions.

Domande guida

- conviene vaccinare uniformemente o mirare gruppi specifici?
- quali blocchi della matrice contano di più?
- un intervento strutturale può abbassare la soglia epidemica?

18. Perché questo è il miglior case study di ancoraggio

Questo modulo è probabilmente il migliore come asse portante del corso per almeno cinque ragioni.

Primo, parte da una rappresentazione molto concreta: un grafo di contatti.

Secondo, porta naturalmente a sistemi di ODE.

Terzo, introduce un criterio spettrale semplice ma potente.

Quarto, consente di confrontare in modo molto chiaro:

- modelli medi;
- simulazioni agent-based;
- interventi strutturali.

Quinto, ha applicazioni immediate a problemi reali di salute pubblica, diffusione sociale e controllo su rete.

19. Conclusione

Le epidemie su reti costituiscono un caso di studio esemplare per i metodi computazionali. In un unico quadro si incontrano:

- struttura topologica;
- dinamica non lineare;
- soglie di invasione;
- simulazione numerica;
- processi stocastici discreti;
- politiche di intervento.

Per questo motivo, il modulo non e' solo un'applicazione delle dinamiche epidemiche, ma un ponte molto efficace tra modellizzazione matematica, analisi numerica e interpretazione interdisciplinare.

20. Bibliografia minima

1. Pastor-Satorras, R., Castellano, C., Van Mieghem, P., and Vespignani, A. Epidemic Processes in Complex Networks.
2. Keeling, M. J., and Eames, K. T. D. Networks and Epidemic Models.
3. Newman, M. Networks: An Introduction.
4. Diekmann, O., Heesterbeek, H., and Britton, T. Mathematical Tools for Understanding Infectious Disease Dynamics.
5. Van Mieghem, P. Epidemic Processes on Complex Networks.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i modelli epidemici su rete discussi nella dispensa.

L'obiettivo e' duplice:

- fornire a chi conosce Python una base quasi immediatamente eseguibile;
- offrire a chi usa altri linguaggi una guida leggibile come pseudocodice operativo.

Per questo motivo il codice e' volutamente elementare:

- poche librerie;
- funzioni corte;
- cicli espliciti;
- strutture dati semplici;
- nessun uso di librerie specialistiche per grafi o integrazione numerica.

La logica generale sara' questa:

1. rappresentare la rete;

2. costruire i modelli mean-field;
3. integrare numericamente le ODE;
4. implementare una simulazione agent-based;
5. confrontare soglia spettrale e dinamica simulata;
6. aggiungere una versione elementare per classi di eta'.

A.1 Librerie minime

Per tutto quello che segue bastano:

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per simulazioni Monte Carlo;
- `math` serve per qualche funzione numerica elementare;
- `statistics` serve per medie semplici;
- `matplotlib.pyplot` serve per i grafici.

Non e' necessario usare `numpy` in una prima implementazione.

A.2 Rappresentare la rete

Il modo piu' semplice e' usare una matrice di adiacenza come lista di liste.

Per esempio, una rete di 5 nodi:

```
A = [
    [0, 1, 1, 0, 0],
    [1, 0, 1, 1, 0],
    [1, 1, 0, 0, 0],
    [0, 1, 0, 0, 1],
    [0, 0, 0, 1, 0]
]
```

Qui:

- $A[i][j] = 1$ se esiste un contatto tra i e j ;
- $A[i][j] = 0$ altrimenti.

Questa rappresentazione e' molto leggibile e sufficiente per reti piccole o medie.

A.2.1 Numero di nodi

```
def number_of_nodes(A):
    return len(A)
```

A.2.2 Grado di un nodo

```
def node_degree(A, i):  
    return sum(A[i])
```

A.2.3 Tutti i gradi

```
def all_degrees(A):  
    degrees = []  
  
    for i in range(len(A)):  
        degrees.append(node_degree(A, i))  
  
    return degrees
```

A.3 SIS mean-field a livello di nodo

Nel modello SIS mean-field, la dinamica di ogni nodo e'

$$\dot{p}_i = -\mu p_i + \beta(1 - p_i) \sum_{j=1}^N a_{ij} p_j.$$

A.3.1 Pressione infettiva locale

```
def infection_pressure(A, p, i):  
    total = 0.0  
  
    for j in range(len(A)):  
        total += A[i][j] * p[j]  
  
    return total
```

A.3.2 Campo dinamico SIS

```
def sis_field(A, p, beta, mu):  
    dp = []  
  
    for i in range(len(A)):  
        h_i = infection_pressure(A, p, i)  
        value = -mu * p[i] + beta * (1.0 - p[i]) * h_i  
        dp.append(value)  
  
    return dp
```

A.3.3 Un passo di Euler

Per integrare numericamente, usiamo Euler esplicito:

$$p_i(t + \Delta t) = p_i(t) + \Delta t, \dot{p}_i(t).$$

```
def euler_step_sis(A, p, beta, mu, dt):
    dp = sis_field(A, p, beta, mu)

    new_p = []

    for i in range(len(p)):
        value = p[i] + dt * dp[i]

        if value < 0.0:
            value = 0.0
        if value > 1.0:
            value = 1.0

        new_p.append(value)

    return new_p
```

A.3.4 Simulazione completa SIS mean-field

```
def simulate_sis_mean_field(A, p0, beta, mu, dt, T):
    p = p0[:]
    history = [p[:]]

    for t in range(T):
        p = euler_step_sis(A, p, beta, mu, dt)
        history.append(p[:])

    return history
```

A.3.5 Prevalenza media nel tempo

La prevalenza media e' la media delle probabilita' di infezione sui nodi:

$$\rho(t) = \frac{1}{N} \sum_{i=1}^N p_i(t).$$

```
def prevalence_from_history(history):
    prevalence = []

    for p in history:
```



```

        prevalence.append(sum(p) / len(p))

    return prevalence

```

A.3.6 Grafico della prevalenza SIS

```

def plot_prevalence(prevalence, title="Prevalenza epidemica"):
    times = list(range(len(prevalence)))

    plt.plot(times, prevalence)
    plt.xlabel("tempo")
    plt.ylabel("prevalenza")
    plt.title(title)
    plt.ylim(0.0, 1.0)
    plt.show()

```

Esempio:

```

A = [
    [0, 1, 1, 0, 0],
    [1, 0, 1, 1, 0],
    [1, 1, 0, 0, 0],
    [0, 1, 0, 0, 1],
    [0, 0, 0, 1, 0]
]

p0 = [0.0, 0.1, 0.0, 0.0, 0.0]

history = simulate_sis_mean_field(
    A=A,
    p0=p0,
    beta=0.8,
    mu=0.5,
    dt=0.01,
    T=2000
)

prevalence = prevalence_from_history(history)
plot_prevalence(prevalence, title="SIS mean-field")

```

A.4 SIR mean-field

Nel modello SIR si hanno tre variabili per nodo:

- $s[i]$ suscettibile;
- $p[i]$ infetto;
- $r[i]$ rimosso.

La dinamica e'

$$\dot{s}_i = -\beta s_i \sum_j a_{ij} p_j,$$

$$\dot{p}_i = \beta s_i \sum_j a_{ij} p_j - \mu p_i,$$

$$\dot{r}_i = \mu p_i.$$

A.4.1 Campo dinamico SIR

```
def sir_field(A, s, p, r, beta, mu):
    ds = []
    dp = []
    dr = []

    for i in range(len(A)):
        h_i = infection_pressure(A, p, i)

        ds_i = -beta * s[i] * h_i
        dp_i = beta * s[i] * h_i - mu * p[i]
        dr_i = mu * p[i]

        ds.append(ds_i)
        dp.append(dp_i)
        dr.append(dr_i)

    return ds, dp, dr
```

A.4.2 Un passo di Euler per SIR

```
def euler_step_sir(A, s, p, r, beta, mu, dt):
    ds, dp, dr = sir_field(A, s, p, r, beta, mu)

    new_s = []
    new_p = []
    new_r = []

    for i in range(len(s)):
        s_i = s[i] + dt * ds[i]
        p_i = p[i] + dt * dp[i]
        r_i = r[i] + dt * dr[i]

        if s_i < 0.0:
```

```

        s_i = 0.0
    if p_i < 0.0:
        p_i = 0.0
    if r_i < 0.0:
        r_i = 0.0

    total = s_i + p_i + r_i

    if total > 0.0:
        s_i /= total
        p_i /= total
        r_i /= total

    new_s.append(s_i)
    new_p.append(p_i)
    new_r.append(r_i)

    return new_s, new_p, new_r

```

A.4.3 Simulazione completa SIR

```

def simulate_sir_mean_field(A, s0, p0, r0, beta, mu, dt, T):
    s = s0[:]
    p = p0[:]
    r = r0[:]

    history_s = [s[:]]
    history_p = [p[:]]
    history_r = [r[:]]

    for t in range(T):
        s, p, r = euler_step_sir(A, s, p, r, beta, mu, dt)

        history_s.append(s[:])
        history_p.append(p[:])
        history_r.append(r[:])

    results = {
        "history_s": history_s,
        "history_p": history_p,
        "history_r": history_r
    }

    return results

```

A.4.4 Prevalenze medie SIR

```
def average_compartment(history):
    averages = []

    for state in history:
        averages.append(sum(state) / len(state))

    return averages
```

A.4.5 Grafico SIR

```
def plot_sir_averages(results):
    avg_s = average_compartment(results["history_s"])
    avg_p = average_compartment(results["history_p"])
    avg_r = average_compartment(results["history_r"])

    times = list(range(len(avg_s)))

    plt.plot(times, avg_s, label="S")
    plt.plot(times, avg_p, label="I")
    plt.plot(times, avg_r, label="R")
    plt.xlabel("tempo")
    plt.ylabel("frazione media")
    plt.title("SIR mean-field su rete")
    plt.ylim(0.0, 1.0)
    plt.legend()
    plt.show()
```

A.5 SEIR mean-field

Nel modello SEIR aggiungiamo la variabile $e[i]$ per gli esposti.

A.5.1 Campo dinamico SEIR

```
def seir_field(A, s, e, p, r, beta, sigma, mu):
    ds = []
    de = []
    dp = []
    dr = []

    for i in range(len(A)):
        h_i = infection_pressure(A, p, i)

        ds_i = -beta * s[i] * h_i
        de_i = beta * s[i] * h_i - sigma * e[i]
```

```

dp_i = sigma * e[i] - mu * p[i]
dr_i = mu * p[i]

ds.append(ds_i)
de.append(de_i)
dp.append(dp_i)
dr.append(dr_i)

return ds, de, dp, dr

```

A.5.2 Un passo di Euler per SEIR

```

def euler_step_seir(A, s, e, p, r, beta, sigma, mu, dt):
    ds, de, dp, dr = seir_field(A, s, e, p, r, beta, sigma, mu)

    new_s = []
    new_e = []
    new_p = []
    new_r = []

    for i in range(len(s)):
        s_i = s[i] + dt * ds[i]
        e_i = e[i] + dt * de[i]
        p_i = p[i] + dt * dp[i]
        r_i = r[i] + dt * dr[i]

        if s_i < 0.0:
            s_i = 0.0
        if e_i < 0.0:
            e_i = 0.0
        if p_i < 0.0:
            p_i = 0.0
        if r_i < 0.0:
            r_i = 0.0

        total = s_i + e_i + p_i + r_i

        if total > 0.0:
            s_i /= total
            e_i /= total
            p_i /= total
            r_i /= total

    new_s.append(s_i)
    new_e.append(e_i)
    new_p.append(p_i)

```

```

        new_r.append(r_i)

    return new_s, new_e, new_p, new_r

```

A.5.3 Simulazione completa SEIR

```

def simulate_seir_mean_field(A, s0, e0, p0, r0, beta, sigma, mu, dt, T):
    s = s0[:]
    e = e0[:]
    p = p0[:]
    r = r0[:]

    history_s = [s[:]]
    history_e = [e[:]]
    history_p = [p[:]]
    history_r = [r[:]]

    for t in range(T):
        s, e, p, r = euler_step_seir(A, s, e, p, r, beta, sigma, mu, dt)

        history_s.append(s[:])
        history_e.append(e[:])
        history_p.append(p[:])
        history_r.append(r[:])

    results = {
        "history_s": history_s,
        "history_e": history_e,
        "history_p": history_p,
        "history_r": history_r
    }

    return results

```

A.6 Approssimazione spettrale della soglia epidemica

La soglia mean-field SIS e'

$$\frac{\beta}{\mu} < \frac{1}{\lambda_{\max}(A)}.$$

Per evitare librerie avanzate, possiamo stimare l'autovalore principale con una iterazione di potenza.

A.6.1 Prodotto matrice-vettore

```
def matrix_vector_product(A, x):
    result = []

    for i in range(len(A)):
        total = 0.0
        for j in range(len(x)):
            total += A[i][j] * x[j]
        result.append(total)

    return result
```

A.6.2 Norma euclidea

```
def vector_norm(x):
    total = 0.0

    for value in x:
        total += value * value

    return math.sqrt(total)
```

A.6.3 Iterazione di potenza

```
def leading_eigenvalue_power_iteration(A, num_steps=100):
    n = len(A)
    x = [1.0 for _ in range(n)]

    for step in range(num_steps):
        y = matrix_vector_product(A, x)
        norm_y = vector_norm(y)

        if norm_y == 0.0:
            return 0.0

        x = [value / norm_y for value in y]

    Ax = matrix_vector_product(A, x)

    numerator = 0.0
    denominator = 0.0

    for i in range(n):
        numerator += x[i] * Ax[i]
        denominator += x[i] * x[i]
```

```

    if denominator == 0.0:
        return 0.0

    return numerator / denominator

```

A.6.4 Soglia teorica

```

def epidemic_threshold_sis(A):
    lambda_max = leading_eigenvalue_power_iteration(A)

    if lambda_max == 0.0:
        return None

    return 1.0 / lambda_max

```

Esempio:

```

threshold = epidemic_threshold_sis(A)
print("Soglia teorica beta/mu <", threshold)

```

A.7 Simulazione agent-based SIS

Ora passiamo alla versione stocastica discreta.

Ogni nodo può essere:

- 0 suscettibile;
- 1 infetto.

A ogni passo discreto:

- un suscettibile si infetta con probabilita' dipendente dai vicini infetti;
- un infetto guarisce con probabilita' $\mu * dt$.

A.7.1 Conta dei vicini infetti

```

def infected_neighbors(A, state, i):
    total = 0

    for j in range(len(A)):
        if A[i][j] == 1 and state[j] == 1:
            total += 1

    return total

```

A.7.2 Un passo di simulazione SIS agent-based

Per un suscettibile con m vicini infetti, una scelta semplice e' usare la probabilita'

$$1 - (1 - \beta dt)^m.$$

```
def sis_agent_based_step(A, state, beta, mu, dt):
    new_state = state[:]

    for i in range(len(state)):
        if state[i] == 0:
            m = infected_neighbors(A, state, i)
            infection_probability = 1.0 - ((1.0 - beta * dt) ** m)

            if random.random() < infection_probability:
                new_state[i] = 1

        elif state[i] == 1:
            recovery_probability = mu * dt

            if random.random() < recovery_probability:
                new_state[i] = 0

    return new_state
```

A.7.3 Simulazione completa SIS agent-based

```
def simulate_sis_agent_based(A, state0, beta, mu, dt, T):
    state = state0[:]
    history = [state[:]]

    for t in range(T):
        state = sis_agent_based_step(A, state, beta, mu, dt)
        history.append(state[:])

    return history
```

A.7.4 Prevalenza agent-based

```
def prevalence_from_agent_history(history):
    prevalence = []

    for state in history:
        prevalence.append(sum(state) / len(state))

    return prevalence
```

A.7.5 Grafico del confronto SIS

```
def plot_two_prevalences(prevalence_1, prevalence_2, label_1, label_2, title):
    times_1 = list(range(len(prevalence_1)))
    times_2 = list(range(len(prevalence_2)))

    plt.plot(times_1, prevalence_1, label=label_1)
    plt.plot(times_2, prevalence_2, label=label_2)
    plt.xlabel("tempo")
    plt.ylabel("prevalenza")
    plt.title(title)
    plt.ylim(0.0, 1.0)
    plt.legend()
    plt.show()
```

Esempio:

```
p0 = [0.0, 0.2, 0.0, 0.0, 0.0]
history_mf = simulate_sis_mean_field(A, p0, beta=0.8, mu=0.5, dt=0.01, T=2000)
prev_mf = prevalence_from_history(history_mf)

state0 = [0, 1, 0, 0, 0]
history_ab = simulate_sis_agent_based(A, state0, beta=0.8, mu=0.5, dt=0.01, T=2000)
prev_ab = prevalence_from_agent_history(history_ab)

plot_two_prevalences(
    prev_mf,
    prev_ab,
    label_1="mean-field SIS",
    label_2="agent-based SIS",
    title="Confronto tra SIS mean-field e agent-based"
)
```

A.8 Ripetere molte simulazioni agent-based

Per avere un confronto corretto, conviene fare medie su molte realizzazioni.

```
def run_many_sis_agent_based(A, state0, beta, mu, dt, T, num_runs):
    all_histories = []

    for run in range(num_runs):
        history = simulate_sis_agent_based(A, state0, beta, mu, dt, T)
        prevalence = prevalence_from_agent_history(history)
        all_histories.append(prevalence)

    return all_histories
```

A.8.1 Media di molte traiettorie

```
def average_trajectories(histories):
    T = len(histories[0])
    mean_history = []

    for t in range(T):
        values_t = []

        for history in histories:
            values_t.append(history[t])

        mean_history.append(statistics.mean(values_t))

    return mean_history
```

Esempio:

```
histories_ab = run_many_sis_agent_based(
    A=A,
    state0=state0,
    beta=0.8,
    mu=0.5,
    dt=0.01,
    T=2000,
    num_runs=50
)

mean_ab = average_trajectories(histories_ab)

plot_two_prevalences(
    prev_mf,
    mean_ab,
    label_1="mean-field SIS",
    label_2="media agent-based",
    title="Confronto tra mean-field e media Monte Carlo"
)
```

A.9 Modello per classi di eta'

Per una estensione semplice a classi di eta', si puo' usare una matrice di contatto C e lavorare su frazioni aggregate per gruppo.

A.9.1 Campo dinamico SIR strutturato per eta'

```
def age_structured_sir_field(C, S, I, R, beta, mu):
    dS = []
```

```

dI = []
dR = []

for a in range(len(C)):
    force = 0.0

    for b in range(len(C)):
        force += C[a][b] * I[b]

    dS_a = -beta * S[a] * force
    dI_a = beta * S[a] * force - mu * I[a]
    dR_a = mu * I[a]

    dS.append(dS_a)
    dI.append(dI_a)
    dR.append(dR_a)

return dS, dI, dR

```

A.9.2 Un passo di Euler

```

def euler_step_age_structured_sir(C, S, I, R, beta, mu, dt):
    dS, dI, dR = age_structured_sir_field(C, S, I, R, beta, mu)

    new_S = []
    new_I = []
    new_R = []

    for a in range(len(S)):
        S_a = S[a] + dt * dS[a]
        I_a = I[a] + dt * dI[a]
        R_a = R[a] + dt * dR[a]

        if S_a < 0.0:
            S_a = 0.0
        if I_a < 0.0:
            I_a = 0.0
        if R_a < 0.0:
            R_a = 0.0

        total = S_a + I_a + R_a

        if total > 0.0:
            S_a /= total
            I_a /= total
            R_a /= total

```

```

        new_S.append(S_a)
        new_I.append(I_a)
        new_R.append(R_a)

    return new_S, new_I, new_R

```

A.9.3 Vaccinazione mirata

Una forma molto semplice di vaccinazione per gruppo è modificare le condizioni iniziali:

```

def vaccinate_initial_susceptibles(S0, R0, vaccination_rates):
    new_S = []
    new_R = []

    for a in range(len(S0)):
        vaccinated = vaccination_rates[a] * S0[a]
        new_S.append(S0[a] - vaccinated)
        new_R.append(R0[a] + vaccinated)

    return new_S, new_R

```

Questa struttura è molto utile per esercizi su strategie di intervento.

A.10 Organizzazione consigliata del file

Per mantenere il codice leggibile, conviene organizzarlo così:

1. import delle librerie;
2. struttura della rete:
 - grado
 - pressione infettiva
3. modelli mean-field:
 - SIS
 - SIR
 - SEIR
4. soglia spettrale:
 - prodotto matrice-vettore
 - power iteration
5. simulazione agent-based:
 - conteggio dei vicini infetti
 - passo di simulazione

- simulazione completa
 - medie Monte Carlo
6. modello per eta':
- campo dinamico
 - vaccinazione
7. blocco finale con esempi.

Per esempio:

```
if __name__ == "__main__":
    A = [
        [0, 1, 1, 0, 0],
        [1, 0, 1, 1, 0],
        [1, 1, 0, 0, 0],
        [0, 1, 0, 0, 1],
        [0, 0, 0, 1, 0]
    ]

    threshold = epidemic_threshold_sis(A)
    print("Soglia teorica beta/mu <", threshold)

    p0 = [0.0, 0.2, 0.0, 0.0, 0.0]
    history_mf = simulate_sis_mean_field(A, p0, beta=0.8, mu=0.5, dt=0.01, T=2000)
    prev_mf = prevalence_from_history(history_mf)

    state0 = [0, 1, 0, 0, 0]
    histories_ab = run_many_sis_agent_based(A, state0, beta=0.8, mu=0.5, dt=0.01, T=2000, n=100)
    mean_ab = average_trajectories(histories_ab)

    plot_two_prevalences(
        prev_mf,
        mean_ab,
        label_1="mean-field",
        label_2="Monte Carlo medio",
        title="Confronto SIS su rete"
    )
```

A.11 Perché questa appendice è utile

Questa appendice è particolarmente utile perché rende visibile una progressione metodologica molto forte:

1. rappresentazione della rete;
2. equazioni differenziali mean-field;
3. criterio spettrale;
4. simulazione agent-based;

5. confronto tra dinamica media e realizzazioni Monte Carlo;
6. estensione strutturata per età.

Questo fa di epidemie su reti un caso di studio estremamente completo.

A.12 Conclusione dell'appendice

La struttura proposta qui è volutamente semplice. Chi conosce Python può implementarla quasi direttamente; chi usa altri linguaggi può leggerla come pseudocodice molto vicino a una traduzione operativa.

Il punto metodologico centrale è che questo modulo permette di confrontare in modo molto chiaro livelli diversi di descrizione:

- rete come struttura dei contatti;
- ODE mean-field come dinamica media;
- simulazione agent-based come realizzazione stocastica;
- matrici di contatto come estensione strutturata per gruppi.